Distance de Jaccard

Rapport de Projet

Élèves :
Mateo Damiani
Yoen Pesquer

Enseignants :
Mr Cristophe Hancart
Mr Arnaud Lefebvre

15 mai 2025

Table des matières

1 Introduction **2**
- **a** Explication du projet . . . 2
- **b** Structure de données utilisées . . . 2
  - **I** Structure d'options . . . 2
  - **II** Structure de contexte . . . 4
  - **III** Structure de distance . . . 5
  - **IV** Structure de cref . . . 6
  - **V** Structure de table de hachage (et fourre-tout) . . . 7
  - **VI** Structure d'arbres binaires valués . . . 8

2 Corps du projet **8**
- **a** Parcours du code principal . . . 8
  - **I** Initialisation des structures . . . 8
  - **II** Lecture des mots . . . 8
  - **III** Calcul de la distance de Jaccard . . . 10
  - **IV** Affichage Simple . . . 11
  - **V** Libération des données . . . 13
- **b** Introduction des options dans le code . . . 13
  - **I** Gestion des options . . . 13
  - **II** L'aide . . . 13
  - **III** Initiales . . . 14
  - **IV** Ponctuations . . . 15
  - **V** Affichage graphique . . . 15
  - **VI** Implantation . . . 17

3 Conclusion **18**
- **a** Différence d'implantation (performance) . . . 18
- **b** Conclusion du projet . . . 19

1 Introduction

a Explication du projet

La distance de Jaccard est un concept mathématique représentant la similarité entre deux ensembles, c'est-à-dire que la distance de Jaccard représente la proportion des éléments qui ne sont pas dans les deux ensembles. La distance de Jaccard peut s'écrire sous cette forme avec A et B deux ensembles :

$$J(A,B) = \begin{cases} 0 & \text{si } A = \emptyset \bigwedge B = \emptyset \\ 1 - \frac{|A \cap B|}{|A \cup B|} & \text{sinon} \end{cases}$$

FIGURE 1 – Formule distance de Jaccard

L'objectif de ce projet va être d'implanter à partir d'un certain nombre d'ensembles (ici des ensembles de mots) donné par l'utilisateur afin de calculer la distance de Jaccard de chacun des ensembles entre eux puis si la sortie n'est pas redirigée vers un fichier, de l'afficher sur le terminal selon les options choisis par l'utilisateur. Deux implantations ont été réalisées, la première utilisant les tables de hachages et la seconde utilisant les arbres binaires de recherche (AVL). Après avoir présenté et analysé le code du projet, on s'attardera sur les modifications de comportements du programme selon les options choisies par l'utilisateur. Et enfin on comparera les deux implantations en temps et en espace grâce aux commandes "time" et "valgrind".

(Toutes fonctions apparaissant dans le rapport seront décrites, mais les spécifications détaillées sont dans les fichiers du projet)

b Structure de données utilisées

I Structure d'options

La structure d'options nommée opts est définie dans op.h et le corps de la structure est présent dans op.c, Il y a quatre champs, trois booléens indiquant si l'option associée a été demandée par l'utilisateur et le size_t "length_max" représentant le paramètre que l'utilisateur aura précisé en appelant l'option -i ou - -initial. La structure opts est initialisée par la fonction opts_empty(), allouant les ressources nécessaires à la structure, puis initialise tous les booléens à faux et "length_max" à 0, un pointeur nul sera renvoyé en cas d'erreur, un pointeur vers l'élément sinon :

Structure d'option

```
struct opts {
  size_t length_max;
  bool ponct;
  bool graph;
  bool isAvl;
}
```

Initialisation de la structure opts

```
opts *opts_empty() {
  opts *p = malloc(sizeof *p);
  if (p == nullptr) {
    return nullptr;
  }
  p->length_max = 0;
  p->ponct = false;
  p->graph = false;
  p->isAvl = false;
  return p;
}
```

La structure est désallouée par la fonction opts_dispose

Désalocation de la structure opts

```
void opts_dispose(opts **p) {
  free(*p);
  p = nullptr;
}
```

Et enfin il y a les requêtes renvoyant un champ précis d'un l'élément, on retrouve : opts_length_max, opts_isPonct, opts_isGraph et opts_isAvl.

II Structure de contexte

La structure de contexte nommée context est définie dans op.h et le corps de la structure est présent dans op.c, La structure représente l'ensemble des fichiers, avec le champ "entry_max" représentant la taille maximale de aref, "cur_pos" représente le nombre courant d'élément dans aref, et enfin le champ aref qui est un tableau contenant tous les fichiers mis en entrée par l'utilisateur. La structure est initialisée par la fonction context_empty() qui va allouer les ressources nécessaires à un élément de type context, "cur_pos" sera initialisée à 0, "entry_max" sera initialisée à la macro constante ENTRY_MAX puis ENTRY_MAX cases sont allouées pour aref, un pointeur nul sera renvoyé en cas d'erreur, un pointeur vers l'élément sinon.

Structure de context

```
struct context {
  size_t entry_max;
  size_t cur_pos;
  char **aref;
};
```

Initialisation de la structure context

```
context *context_empty() {
  context *p = malloc(sizeof *p);
  if (p == nullptr) {
    return nullptr;
  }
  p->entry_max = ENTRY_MAX;
  p->cur_pos = 0;
  p->aref = malloc(sizeof(char *) * ENTRY_MAX);
  if (p->aref == nullptr) {
    free(p);
    return nullptr;
  }
  return p;
}
```

La structure est désallouée par context_dispose.

Désalocation structure context

```
void context_dispose(context **cntxptr) {
  free((*cntxptr)->aref);
  free(*cntxptr);
  cntxptr = nullptr;
}
```

On retrouve également les requêtes context_entry, context_pos et context_aref.

III Structure de distance

La structure de distance nommée dist est définie dans main.c et le corps de la structure est présent dans main.c La structure dist contient trois champs, un size_t "nmemb" représentant le nombre de texte différents, puis deux tableaux représentant l'union et l'intersection de deux textes, chacun des tableaux possède une taille équivalente au nombre de couples possibles dans un ensemble de n éléments :

Structure de distance

```
struct dist {
  float *uni;
  float *inter;
  size_t nmemb;
};
```

$$\frac{n * (n - 1)}{2}$$

FIGURE 2 – Formule nombres de couples

Un élément de type dist est initialisé par la fonction dist_init nécessitant un élément de type context afin de récupérer le nombre de textes utilisé dans le programme.

Initialisation de la structure dist

```
dist *dist_init(context *lfiles) {
  dist *p = malloc(sizeof *p);
  if (p == nullptr) {
    return nullptr;
  }
  size_t nbr_couples = (lfiles->cur_pos * (lfiles->cur_pos - 1)) / 2;
  p->uni = calloc(nbr_couples, sizeof *(p->uni));
  p->inter = calloc(nbr_couples, sizeof *(p->inter));
  p->nmemb = lfiles->cur_pos;
  if ((p->uni == nullptr) || (p->inter == nullptr)) {
    free(p->uni);
    free(p->inter);
    free(p);
    return nullptr;
  }
  return p;
}
```

Les ressources sont libérés en utilisant la fonction dist_dispose :

Désalocation de la structure de distance

```
void dist_dispose(dist **pp) {
  if (*pp == nullptr) {
    return;
  }
  free((*pp)->uni);
  free((*pp)->inter);
  free(*pp);
  *pp = nullptr;
}
```

IV Structure de cref

Les deux types de données (table de hachage et AVL) contiendront des éléments de type cref contenant un const char * représentant un mot et un tableau de booléens décrivant l'appartenance du mot à un des textes (si la case 0 du tableau est "True" cela équivaut à dire que le mot est présent dans le 1er texte).

Structure cref

```
struct cref {
  const char *ref;
  bool *aref;
};
```

Un élément de type cref est initialisé par la fonction cref_empty

Initialisation de la structure de cref

```
cref *cref_empty() {
  cref *p = malloc(sizeof *p);
  if (p == nullptr) {
    return nullptr;
  }
  p->aref = nullptr;
  p->ref = nullptr;
  return p;
}
```

Les ressources sont libérés en utilisant la fonction cref_free :

Initialisation de la structure de cref

```
int cref_free(cref *ptr) {
  if (ptr == nullptr) {
    return 0;
  }
  free((char *) ptr->ref);
  free(ptr->aref);
  free(ptr);
  return 0;
}
```

V Structure de table de hachage (et fourre-tout)

La structure de table de hachage est définie dans le fichier hashtable.h tandis que le fourre-tout est défini dans holdall.h, les fichiers sources ont été récupérés du TP1. Les structures sont initialisées respectivement par hashtable_empty et holdall_empty.

Pour la table de hachage, la fonction de hachage str_hashfun sera utilisée. Un premier holdall contiendra les mots et l'autre les tableaux de booléens pour réaliser l'affichage.

VI Structure d'arbres binaires valués

La structure d'arbre binaire est définie dans bst.h, les fichiers du dossier avl ont été récupérés du TP5. La structure est initialisée par bst_empty. La fonction de comparaison compar est utilisée pour le tri de l'arbre.

Fonction de comparaison

```
int compar(const void *x, const void *y) {
  const cref *cx = (const cref *) x;
  const cref *cy = (const cref *) y;
  return strcoll(cx->ref, cy->ref);
}
```

2 Corps du projet

a Parcours du code principal

I Initialisation des structures

Au début du programme, une boucle est réalisée vérifiant si l'aide est demandée, ensuite, toutes les structures nécessaires à la fonction stdin_read_set sont allouées, en cas d'erreur d'allocation, on renvoie au label error_capacity menant à la libération de toutes les ressources précédemment allouées et au renvoie d'un message d'erreur. Les autres structure sont définies à nullptr pour simplifier les libérations en cas d'erreurs. C'est également à ce moment-là que les options sont lus grâce à la fonction stdin_read_set, cette fonction va parcourir toutes les informations données par l'utilisateur (argv) en début de programme afin d'enregistrer les options choisies mais aussi les fichiers donnés (ou le souhait d'utiliser l'entrée standard). Il y a de nombreux cas d'erreurs, si l'option est inconnu, en cas de problème d'arguments notamment pour l'option "initial" (valeur manquante ou valeur incorrecte), potentielle erreur d'allocation due à l'ajout des fichiers dans le contexte. La fonction stdin_read_set va parcourir argv puis comparer la chaîne avec une des options possibles (d'abord une comparaison pour savoir si l'option est longue ou courte puis avec les différentes macro constantes d'options définies dans le fichier op.h). Après cela, une ultime vérification est faite afin de vérifier que le nombre de fichiers donné par l'utilisateur soit au moins supérieur ou égal à 2. Cette vérification est essentielle pour la réalisation du calcul de la distance de Jaccard.

II Lecture des mots

Grâce au champ "cur_pos" de la structure de contexte, on peut connaître le nombre de textes à lire. Pour chaque élément de lfiles (structure de contexte), un test est réalisé si il s'agit d'un fichier ou de l'entrée standard. Le fichier est lu caractère par caractère avec la fonction fgetc et une chaîne de caractère est allouée dynamiquement pour la lecture des mots. La fin du mot est déterminée par la fonction is_separator. Quand le mot est terminé, la fonction

end_word est appelée, ajoutant le caractère '\0' à la fin du mot, puis alloue pour le mot si il n'est pas déjà présent et l'ajoute à la table de hachage ou à l'arbre binaire selon l'implantation choisie par l'utilisateur. La fonction next_word est ensuite appelée, permettant de passer directement au mot suivant que l'on soit à la fin ou au milieu d'un mot (option -i).

Fonction de mot suivant

```
void next_word(opts **optr, FILE *file) {
  bool sep_found = false;
  int c;
  while ((c = fgetc(file)) != EOF) {
    if (is_separator(optr, c)) {
      sep_found = true;
    } else if (sep_found) {
      ungetc(c, file);
      return;
    }
  }
}
```

next_word fonctionne de pair avec is_separator, qui lui retourne si le caractère c est un séparateur, de type isspace et ispunct si l'option -p ou –punct est activée :

Fonction de vérification de séparateur

```
bool is_separator(opts **optr, int c) {
  if (isspace(c)) {
    return true;
  }
  if (opts_isPonct(optr) && ispunct(c)) {
    return true;
  }
  return false;
}
```

Enfin, les cas de fin de lecture sont testés que ce soit pour les fichiers ou l'entrée standard (avec "pos" le pos-ième texte lu) :

Extrait du code main.c : Fin de lecture

```
if (file != stdin) {
  if (!feof(file)) {
    goto error_read;
  }
  if (fclose(file) != 0) {
    goto error_close;
  }
} else {
  printf("--- ends reading for #%zu FILE\n", pos + 1);
  rewind(file);
}
```

III Calcul de la distance de Jaccard

La fonction dist_calc parcours tous les couples de fichiers sur ptr->aref et actualise les tableaux de "dt" en fonction des valeurs des cases du couples, Si l'une des cases vaut "True", la valeur de la case correspondante à ce couple de dt->uni augmente de 1, Si les 2 cases valent "True", alors la valeur de la case correspondante à ce couple de dt->inter augmente également de 1.

Fonction de calcul de distance

```
int dist_calc(dist *dt, cref *ptr) {
  if (dt == nullptr || ptr == nullptr) {
    return -1;
  }
  size_t p1 = 0;
  size_t p2 = 1;
  size_t dpos = 0;
  while (p1 < (dt->nmemb - 1)) {
    if (ptr->aref[p1] && ptr->aref[p2]) {
      dt->inter[dpos] += 1;
    }
    if (ptr->aref[p1] || ptr->aref[p2]) {
      dt->uni[dpos] += 1;
    }
    p2 += 1;
    if (p2 == dt->nmemb) {
      p1 += 1;
      p2 = p1 + 1;
    }
    dpos += 1;
  }
  return 0;
}
```

IV Affichage Simple

Pour l'affichage, on supposera que l'affichage simple (sans l'option) a été demandé par l'utilisateur, l'affichage détaillé sera expliqué dans le 2.b.V. La fonction parcourt sur les tableaux uni et inter de "dt", elle effectue la formule de la distance de Jaccard pour un couple puis affiche les noms des textes associés au couple stockés dans "lfiles" suivi du résultat :

Fonction d'affichage simple

```
int repr_simple(dist *dt, context *lfiles) {
  size_t nbr_couples = (lfiles->cur_pos * (lfiles->cur_pos - 1)) / 2;
  size_t i = 0;
  size_t k = 1;
  size_t j = 0;
  size_t compteur = 0;
  while (i < nbr_couples) {
    if (k > (lfiles->cur_pos) - j - 1) {
      compteur++;
      k = 1;
      j++;
    }
    const char *file1
      = is_std_file(lfiles->aref[compteur]) ? STD_NAME :
      ↪  lfiles->aref[compteur
        ];
    const char *file2
      = is_std_file(lfiles->aref[k + j]) ? STD_NAME : lfiles->aref[k + j];
    if (dt->uni[i] == 0) {
      float jacard = 0;
      printf("%s\t%s\t%.4f", file1, file2, jacard);
    } else {
      float jacard = 1.f - ((float) (dt->inter[i])) / ((float)
      ↪  (dt->uni[i]));
      printf("%s\t%s\t%.4f", file1, file2, jacard);
    }
    printf("\n");
    i++;
    k++;
  }
  return 0;
}
```

```
lesmiserables.txt        domjuan.txt      0.9535
```

FIGURE 3 – Affichage simple

V Libération des données

A la fin du programme, après avoir défini l'entier de résultat "r" à EXIT_SUCCESS, on renvoie vers le label dispose s'occupant de libérer l'intégralité des structures utilisées (selon l'implantation). Ce label est également utilisé pour les labels d'erreurs afin de permettre la différenciation de plusieurs cas d'erreurs. Enfin, le résultat "r" est renvoyé et le programme se termine.

b Introduction des options dans le code

I Gestion des options

Les options sont lus au travers de la fonction stdin_read_set,
cette fonction comparera l'entrée utilisateur avec toutes les macro constantes liées aux options (commençant par OPT) afin de déterminer les options choisies. En cas de fichier, un appel à la fonction add_file sera effectué, ajoutant ainsi le texte à la structure de contexte tout en mettant à jour ses champs. Cet appel nécessite potentiellement des allocations, un cas d'erreur a été prévu à cet effet via le label "error_capacity", si une options valide est détectée, stdin_read_set met à jour la structure d'option, sinon si une option non reconnu est lu stdin_read_set renvoie un code d'erreur.
(code : voir fichier op.c ligne 154-297)

II L'aide

Au tout début du programme, un premier parcours de argv est réalisé afin de détecter si l'aide est appelée. Ce même parcours est réalisé un peu plus tard dans la fonction stdin_read_set. Ainsi, cette vérification ne serait qu'une perte de temps mais la fonction stdin_read_set nécessite au minimum les allocations de la structure d'option et de contexte. Hors, l'aide doit stopper le programme, ces allocations seraient donc purement inutiles. Cela est donc un choix de programmation entre un programme plus rapide ou un nombre minimal d'allocation pour l'aide. Si l'aide est appelée, la fonction print_help est réalisée, définit dans op.c elle contient l'ensemble des chaînes de l'aide puis appelle la fonction print_str sur chacune d'elle. La fonction print_str va utiliser la bibliothèque ioctl afin de récupérer la longueur du terminal et de réaliser un affichage mot par mot. Après l'appel à print_help, le programme s'arrête puis renvoie EXIT_SUCCESS.

Fonction d'affichage de l'aide

```
static void print_str(const char *text, size_t gap) {
  printf("\n");
  size_t nbrcol = INIT_LENGTH_WORD;
  struct winsize w;
  if (ioctl(STDOUT_FILENO, TIOCGWINSZ, &w) == 0) {
    nbrcol = (size_t) w.ws_col;
  }
  size_t taille_ligne = 0;
  size_t taille_mot = 0;
  const char *start_word = text;
  const char *actual_char = text;
  size_t len = strlen(text);
  printf("%*s", REPR_TAB * (int) (gap), " ");
  for (size_t i = 0; i <= len; i++) {
    if (text[i] == ' ' || text[i] == '\0') {
      taille_ligne += taille_mot + 1;
      if (taille_ligne > nbrcol - TERMINAL_MARGIN) {
        printf("\n");
        printf("%*s", REPR_TAB * (int) (gap + 1), " ");
        taille_ligne = taille_mot + REPR_TAB * (gap + 1);
      }
      for (size_t k = 0; k < taille_mot + 1; k++) {
        printf("%c", start_word[k]);
      }
      start_word = actual_char;
      taille_mot = 0;
    } else {
      taille_mot++;
    }
    actual_char++;
  }
  printf("\n");
}
```

III Initiales

Lors de la lecture des options dans stdin_read_set, après détection de l'option, son argument est obtenu en lisant la chaîne juste après les macro constantes OPT_LONG et LONG_INIT ou l'élément suivant de argv (ainsi -i=3 et -i= 3 fonctionne) et enfin la validité de l'argument est vérifié par la fonction isInt, l'option n'a d'importance qu'à la lecture des mots, un size_t i est défini à 0 et traque la longueur actuelle du mot courant, si i est égal au champ length_max de la structure d'options (argument de l'option -i) alors la fonction end_word est appelée terminant prématurément le mot.

Extrait du code main.c

```
if (i == imax) {
  if (end_word(optr, lfiles, pos, cur, &max, w,
      ht, has, hasbl, bt, cref_w) == -1) {
    goto error_capacity;
  }
  ungetc(c, file);
  next_word(&optr, file);
  cur = 0;
  i = 0;
}
```

IV Ponctuations

L'option de ponctuation si activé, indiquera au programme de considérer les symboles de ponctuations comme un type isspace.
Lors des appels à la fonction is_separator (afin de tester si le mot est terminé), le champ "isPonct" de la structure d'options est testé afin de prendre la ponctuation en compte ou non.

Fonction is_separator

```
bool is_separator(opts **optr, int c) {
  if (isspace(c)) {
    return true;
  }
  if (opts_isPonct(optr) && ispunct(c)) {
    return true;
  }
  return false;
}
```

V Affichage graphique

Lors de la partie de l'affichage, si le champ isGraph de la structure d'options est vraie, alors un apply_context est réalisé sur le fourre-tout (ou sur l'arbre avec la fonction bst_dft_infix_apply_context) avec pour :
- HashTable : la fonction intermediary_repr_graphic (elle appelle la fonction repr_graphic mais elle permet d'uniformiser les implantations)
- AVL : repr_graphic

repr_graphic prend un contexte et un élément de type cref. Grâce au champ cur_pos du contexte, un parcours du tableau de booléen de cref est réalisé afin de réaliser un affichage du mot selon son appartenance aux différents textes.

Fonction d'affichage graphique

```c
int repr_graphic(context *lfiles, cref *ptr) {
  int r = 0;
  printf("%s", ptr->ref);
  size_t k = 0;
  while (k < lfiles->cur_pos) {
    ptr->aref[k] == true ? printf("\tX") : printf("\t-");
    k += 1;
  }
  printf("\n");
  return r;
}
```

```
        lesmiserables.txt       domjuan.txt
l'impiete        -       X
content;         -       X
outrages,        -       X
deshonorees,     -       X
seduites,        -       X
violees,         -       X
offense,         -       X
sens-je?         -       X
renvoie,         -       X
STATUE, -       X
suis-moi.        -       X
repentir.        -       X
jetez-vous       -       X
```

FIGURE 4 – Affichage graphique

VI Implantation

En fonction de si le champ "isAvl" de la structure d'option vaut "True" ou "False", le programme choisira une implémentation différente. Si l'option n'a pas été précisé par l'utilisateur, l'implantation en HashTable sera utilisé.

- Si isAvl = True :

  Alloue et utilise l'implémentation des AVL.

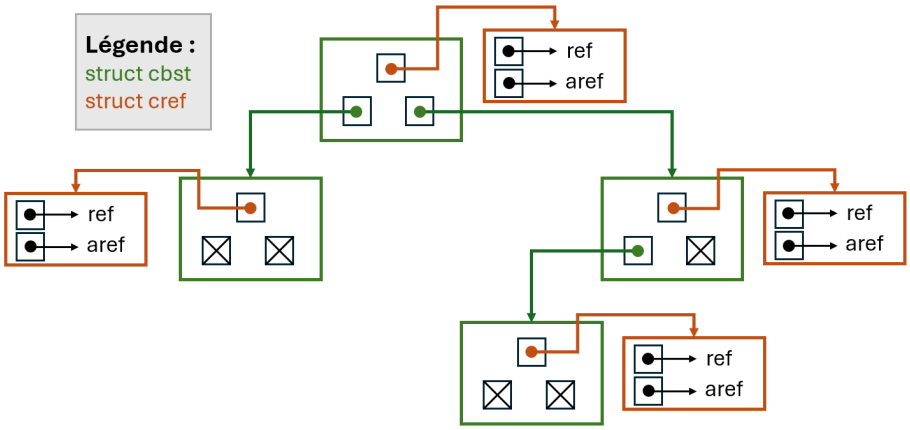

FIGURE 5 – Représentation graphique de la structure avec les AVL

- Si isAvl = False :

  Alloue et utilise l'implémentation des HashTable.

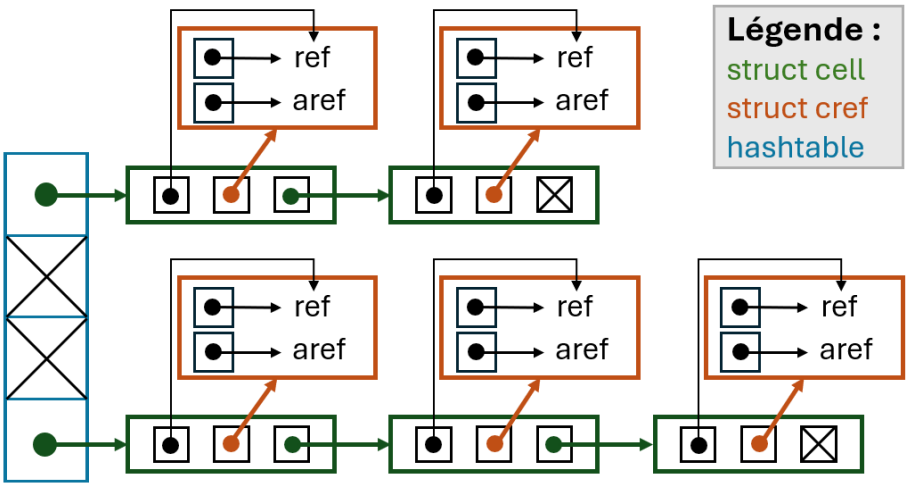

FIGURE 6 – Représentation graphique de la structure avec les Hashtable

3 Conclusion

a Différence d'implantation (performance)

L'objectif est de réaliser une série de tests afin de comparer les implantations en tables de hachage et en AVL (les fonctions time et valgrind seront utilisées).
Les tests ont été réalisés sur la même machine.

Test 1 : lesmiserables.txt et donjuan.txt :

Distance de Jaccard calculé vaut environ 0.9535

Time

- Table de hachage : 0,566s
- AVL : 1.797s

Valgrind

- Table de hachage : 369 841 allocs
- AVL : 246 559 allocs

Test 2 : lesmiserables.txt et lesmiserables.txt :

Distance de Jaccard calculé vaut 0

Time

- Table de hachage : 1.015s
- AVL : 3.189s

Valgrind

- Table de hachage : 361 501 allocs
- AVL : 240 999 allocs

Test 3 : abeeeillmrsss.txt et sssrmllieeeba.txt :

Distance de Jaccard calculé vaut 0

Time

- Table de hachage : 0.955s
- AVL : 3.962s

Valgrind

- Table de hachage : 361 501 allocs
- AVL : 240 999 allocs

Ici les différences sont claires, l'implantation en table de hachage est bien plus rapide que celle en AVL (environ 3 fois plus rapides) mais bien plus coûteuse en espace (environ 1.5 fois plus coûteuses). La structure d'AVL est bien plus coûteuse en temps car il y a des possibles rotations nécessaires pour garder un parcours de l'arbre en log2 (n) après chaque ajout. Tandis que les tables de hachages nécessitent plus d'espace pour les cellules de la table.

b Conclusion du projet

Pour conclure, les deux implantations nous permettent de mettre en valeur un choix commun en informatique, celui de choisir entre un programme plus rapide mais nécessitant plus d'espace ou un programme plus long mais prenant moins d'espace. Ici, cette décision revient à l'utilisateur de choisir l'implémentation qui lui convient le mieu pour réaliser ce dont il a besoin.